

第二模块：MCU体系结构与异常模型（内核级超详细版

• Cortex-M 方向）

本模块属于真正底层内容。

目标： - 理解 MCU 上电后 CPU 到底做了什么 - 理解异常（Exception）与中断的本质 - 理解硬件自动压栈与返回机制

本模块不再讲抽象概念，而是直接进入 **寄存器级执行模型**。

一、MCU启动流程（Reset 到 main 真实路径）

1.1 上电后 CPU 做的第一件事

Cortex-M 上电后不会执行 C 代码。

CPU流程：

1) 读取地址 `0x00000000` 的值 → 初始化 MSP) 读取地址
`0x00000004` 的值 → Reset_Handler 地址 3) PC跳转 Reset_Handler

这就是：

👉 向量表（Vector Table）

1.2 向量表真实结构

```
[0] 初始栈地址
[1] Reset_Handler
[2] NMI_Handler
[3] HardFault_Handler
...
```

本质：

函数指针数组。

1.3 Reset_Handler 内部真正做什么

典型流程：

- 复制 .data 段 (Flash → RAM)
- 清零 .bss 段
- 调用 SystemInit()
- 跳转 main()

重要结论：

👉 main 不是程序入口。

二、Cortex-M 寄存器模型（必须掌握）

核心寄存器：

- R0 - R15：通用寄存器
 - SP：栈指针
 - LR：返回地址
 - PC：程序计数器
 - xPSR：状态寄存器
-

2.1 双栈模型（MSP 与 PSP）

Cortex-M 有两个栈：

- MSP：中断默认使用
- PSP：线程/任务使用

RTOS就是利用这个机制。

三、异常（Exception）执行模型（核心重点）

很多人说“中断函数被调用”，其实不准确。

真实发生的是：

👉 CPU 强制修改 PC。

3.1 异常触发时 CPU 自动压栈内容

硬件自动压入：

```
R0  
R1  
R2  
R3  
R12  
LR  
PC  
xPSR
```

这叫：

👉 Exception Stack Frame

注意：

不是编译器保存，是硬件完成。

3.2 为什么 RTOS 只保存 R4 - R11

原因：

异常已经自动保存 R0 - R3。

RTOS只需要补充剩余寄存器。

3.3 EXC_RETURN（很多人完全不知道）

异常返回时 LR 不是普通地址，而是特殊值。

例如：

```
0xFFFFFFFF9
```

它包含信息：

- 返回使用 MSP 还是 PSP
 - 是否浮点上下文
-

四、NVIC 与优先级机制（底层理解）

NVIC 负责：

- 中断使能
 - 优先级
 - 嵌套控制
-

4.1 PRIMASK / BASEPRI / FAULTMASK 区别

PRIMASK：

关闭所有可屏蔽中断。

BASEPRI：

只屏蔽低优先级中断。

FAULTMASK：

几乎全部关闭（很少用）。

4.2 Tail-Chaining（尾链优化）

当一个中断结束马上有下一个：

CPU不会恢复线程再进入中断。

优势：

减少栈操作，提高效率。

4.3 Late Arrival（迟到机制）

更高优先级中断在压栈过程中到来：

CPU直接切换到更高优先级。

五、异常返回流程（真正底层行为）

执行：

BX LR

CPU检测 EXC_RETURN:

- 自动弹栈
- 恢复 PC
- 恢复 xPSR

六、HardFault 深度理解（不是只会看报错）

常见原因:

- 未对齐访问
- 空指针
- 栈溢出
- 执行非法地址

定位思路:

读取自动压栈的 PC。

七、高频底层考点（标准答案）

Q 1：中断发生时 CPU 做了什么？

标准答案:

CPU自动压栈 8 个寄存器，然后跳转向量表。

Q 2：为什么 RTOS 只保存 R 4 -R 11？

标准答案:

因为异常入口已经自动保存 R 0 -R 3。

Q 3：EXC_RETURN 是什么？

标准答案:

异常返回控制码，指示使用哪种栈与模式。

八、工程注意事项（真正经验）

- 1) 不要在 ISR 里做复杂逻辑。
- 2) 优先级不要全部设最高。
- 3) 理解自动压栈是调试HardFault关键。

九、深度练习（强烈建议）

- 手动解析一次异常栈帧
- 打印 LR 观察 EXC_RETURN
- 用汇编查看异常入口代码

完成本模块后，你将真正理解：中断不是函数调用，而是CPU异常状态切换。